H100 GPU Machine File — Comprehensive Documentation

File: `H100_Helma_GPU.yml` **Device:** NVIDIA H100 (Hopper, Compute Capability 9.0) **Cluster:** Helma, NHR@FAU, CUDA 12.8 **Generated:** May 2026 **Method:** Nsight Compute (ncu) hardware-counter profiling of custom CUDA benchmark

---

Table of Contents

1. Purpose and Context
2. System Description
3. Benchmarking Methodology
4. File Format — Section by Section
5. Comparison with CPU Machine File
6. Measurement Data — Full Results Table
7. Key Findings and Interpretation
8. Relationship to kerncraft
9. Limitations and Future Work

---

1. Purpose and Context

kerncraft is a performance modelling tool developed at RRZE (FAU) that predicts the execution time of loop kernels on a given hardware platform. Its primary input is a machine file — a YAML document describing the hardware's memory hierarchy, bandwidth, latency, and in-core execution capabilities. Machine files exist for dozens of CPU models, but none existed for Nvidia GPU hardware.

This document and the accompanying YAML file represent the first attempt to construct a kerncraft-compatible machine file for the NVIDIA H100 GPU on the Helma HPC cluster. The goal is to enable roofline-based performance modelling of GPU kernels using the same framework used for CPU kernels at RRZE.

---

2. System Description

Hardware

Parameter	Value	Source
GPU model	NVIDIA H100	cudaDeviceProp.name

Parameter	Value	Source
Architecture	Hopper	Compute Capability 9.0
Memory variant	94 GB HBM (special NVL-like variant)	cudaDeviceProp.totalGlobalMem
Streaming Multiprocessors	132	cudaDeviceProp.multiProcessorCount
GPU clock	1.98 GHz	cudaDeviceProp.clockRate
Memory clock	1.593 GHz	cudaDeviceProp.memoryClockRate
Memory bus width	6016 bits	cudaDeviceProp.memoryBusWidth
Theoretical peak BW	2395.87 GB/s	2 × 1.593 GHz × 6016/8 bytes
L2 cache	63 MB	cudaDeviceProp.l2CacheSize
Shared memory per SM	228 kB	cudaDeviceProp.sharedMemPerMultiprocessor
Warp size	32 threads	cudaDeviceProp.warpSize
Max threads per SM	2048	cudaDeviceProp.maxThreadsPerMultiProcessor
Max threads per block	1024	cudaDeviceProp.maxThreadsPerBlock
Registers per SM	65536	cudaDeviceProp.regsPerMultiprocessor
ECC	Enabled	cudaDeviceProp.ECCEnabled
Concurrent kernels	1	cudaDeviceProp.concurrentKernels
PCIe bandwidth	56.38 GB/s	cuda-memcpy benchmark

Note on memory bus width: The standard H100 SXM5 uses a 5120-bit bus (80 GB HBM3). This Helma variant has a 6016-bit bus and 100.2 GB total memory, making it a custom high-memory variant with a proportionally higher theoretical peak bandwidth of 2395.87 GB/s compared to the standard 3350 GB/s for H100 SXM5.

Software

Component	Version
CUDA Toolkit	12.8.0
nvcc	Release 12.8, V12.8.61
Nsight Compute (ncu)	CUDA 12.8
Compute capability target	sm_90

---

3. Benchmarking Methodology

Tools used

Four benchmark tools were run and their outputs combined:

Tool	What it measures	How it was used
collect_gpu_info.cu	Hardware parameters via cudaDeviceProp	One-time run; fills all topology fields
gpu-benches (RRZE)	Cache BW, latency, PCIe, operation latency	Separate runs; fills memory hierarchy fields
MixBench	Roofline ceiling sweep (DP/SP/HP)	One run; fills roofline ceilings
BabelStream	Stream kernels at fixed block=256	Validation cross-check only
Custom gpu_bench_ncu + ncu	5 kernels × 6 block sizes × 3 array sizes	Primary source of all measurement data

Custom benchmark kernels

Five double-precision streaming kernels were written in CUDA (gpu_bench_ncu.cu), each using a grid-stride loop to process arrays of configurable size at configurable block size:

Kernel	Formula	Reads	Writes	FLOPs/elem	Arithmetic Intensity
load	`sum += A[i]`	1	0	1 (reduction)	0.125 FLOP/byte
copy	`B[i] = A[i]`	1	1	0	0.0 FLOP/byte
update	`A[i] += scalar`	1 (RW)	1	1	0.0625 FLOP/byte
daxpy	`A[i] += s * B[i]`	1+1 (RW)	1	2	0.0833 FLOP/byte
triad	`A[i] = B[i] + s*C[i]`	2	1	2	0.0833 FLOP/byte

All five kernels are deeply memory-bound. The roofline ridge point for DP on this H100 is ~14.6 FLOP/byte (from MixBench), while the highest arithmetic intensity in our kernel set is 0.125 FLOP/byte. This means bandwidth — not compute — is always the performance ceiling.

ncu hardware counter profiling

Instead of wall-clock timing (which measures elapsed time including OS overheads),
the benchmark uses Nsight Compute (ncu) to collect hardware performance counters directly from the GPU silicon. The following counters were collected per kernel invocation:

Counter	Meaning
`dram__bytes_read.sum`	Actual bytes read from HBM memory
`dram__bytes_write.sum`	Actual bytes written to HBM memory
`gpu__time_duration.sum`	Kernel execution time in nanoseconds
`l1tex__m_xbar2l1tex_read_bytes_mem_lg_op_ld.sum`	Bytes read from L2 into SMs
`l1tex__m_l1tex2xbar_write_bytes_mem_lg_op_st.sum`	Bytes written from SMs into L2

Bandwidth was computed as: `(read_bytes + write_bytes) / (time_ns × 1e-9) / 1e9` GB/s.

ncu uses multi-pass replay: when all requested metrics cannot be collected in a single kernel execution, ncu automatically re-runs the kernel and combines the results. Each ncu invocation produced two data points. With 5 repetitions per configuration, each measurement entry has up to 10 data points.

Sweep dimensions

Dimension	Values	Rationale
Block size	32, 64, 128, 256, 512, 1024	Full range of valid block sizes on H100
Array size (L2)	4 MB/array	Fits inside 63 MB L2 (3 arrays × 4 MB = 12 MB < 63 MB)
Array size (HBM moderate)	134 MB/array	Overflows L2, HBM-bound
Array size (HBM saturated)	536 MB/array	Deeply HBM-bound, stable saturation
Kernels	5	load, copy, update, daxpy, triad
Repetitions	5 per config	× ~2 ncu passes = ~10 data points

Total configurations: 5 kernels × 6 block sizes × 3 array sizes = 90 combinations.

---

4. File Format — Section by Section

4.1 Hardware header

```
model name: NVIDIA H100
model type: NVIDIA Hopper architecture
compute capability: '9.0'
sockets: 1
SMs: 132
```

Describes the physical device. `SMs` is the GPU equivalent of `cores per socket` in the

CPU machine file. Unlike CPUs, all 132 SMs always participate in every kernel launch — there is no concept of "using fewer SMs" in the way a CPU can use fewer cores. The block size dimension in the measurements captures the effective occupancy instead.

4.2 FLOPs per SM

```
FLOPs per SM:
  DP: {FMA: 64, ADD: 64, MUL: 64, total: 128}
```

Each SM on the H100 Hopper architecture has 64 FP64 FMA units, contributing 128 FP64 operations per cycle (each FMA = 1 multiply + 1 add). At 1.98 GHz with 132 SMs:

Peak DP throughput = 128 × 1.98 GHz × 132 = **33.5 TFLOP/s** (theoretical) MixBench measured: **31.4 TFLOP/s** (93.7% efficiency)

The `FMA: 64` means 64 FMA units per SM. In the CPU file this field is `FLOPs per cycle` per core. The GPU version uses `FLOPs per SM` to make the scope explicit.

4.3 Memory hierarchy

```
memory hierarchy:
- level: L1
  size per group: 228.00 kB      # actual measured (not 256 kB spec)
  groups: 132
  upstream throughput: [shared memory, full-duplex]
- level: L2
  size per group: 63.00 MB       # nominal; effective ~40 MB
  groups: 1
  upstream throughput: [HBM bandwidth, half-duplex]
- level: HBM
  size per group: ~
  groups: 1
  upstream throughput: [full socket memory bandwidth, half-duplex]
```

Three levels are defined: L1 (per-SM), L2 (shared), and HBM (main memory). The GPU has no L3 cache. Key differences from the CPU memory hierarchy:

- **L1**: 228 kB per SM (configured as shared memory; different from the CPU L1 which is a true hardware cache). The CPU file would have sets/ways/cl_size describing a set-associative cache; the GPU L1 is effectively programmer-managed shared memory.
- **L2**: Shared across all 132 SMs. In the CPU file each level has `cores per group` = 1 (each core has its own L2) or `cores per group` = N (shared LLC). Here `groups: 1` means a single shared L2.
- **HBM**: Equivalent to `MEM` in the CPU file. `size per group: ~` (null) because the GPU memory is not divided into groups — all SMs share the full 94 GB.

Measured bandwidth values from gpu-benches:

Level	Aggregate BW	Per-SM BW	Source
L1	33,604 GB/s	~255 GB/s	gpu-cache
L2	12,759 GB/s	shared	gpu-l2-cache
HBM	2,229 GB/s	shared	gpu-l2-cache

4.4 Benchmark kernels section

```
benchmarks:
  kernels:
    copy:
      FLOPs per iteration: 0
      read streams:  {bytes: 8.00 B, streams: 1}
      write streams: {bytes: 8.00 B, streams: 1}
```

This section describes the memory access pattern of each kernel in the same format as the CPU machine file. kerncraft uses this to compute arithmetic intensity and classify kernels as memory-bound or compute-bound on the roofline. `bytes: 8.00 B` means one double-precision element (8 bytes) per stream per iteration.

4.5 Measurements section

The measurements section is the primary deliverable. It contains 18 entries:

```
L2_4MB_block{32,64,128,256,512,1024}     — L2-resident workload
HBM_134MB_block{32,64,128,256,512,1024}  — moderate HBM workload
HBM_536MB_block{32,64,128,256,512,1024}  — saturated HBM workload
```

Each entry has the following structure:

```
HBM_536MB_block256:
  total size: 536.00 MB       # total array size per stream
  block size: 256             # CUDA threads per block (the GPU "core count"
analog)
  grid size: 262144           # CUDA blocks per grid (= array_size / block_size)
  results:
    copy:
      HBM:     [2170.18 GB/s, 2167.43 GB/s, ...]   # raw HBM bandwidth readings
      L2:      [2219.06 GB/s, 2215.98 GB/s, ...]   # raw L2 bandwidth readings
    update:
      HBM:     [2143.40 GB/s, ...]
      L2:      [2198.13 GB/s, ...]
      GFLOP/s: [137.38 GFLOP/s, ...]               # only for FLOPs > 0 kernels
```

HBM vs L2 in measurements: For HBM-bound arrays (134 MB and 536 MB), the `HBM` values represent actual bytes transferred to/from HBM memory (dram__bytes counters). The `L2` values represent bytes that crossed the L2-to-SM boundary. For HBM-bound workloads these two should be proportional. For L2-bound workloads (4 MB array), the `HBM` column shows HBM traffic close to zero (L2 serves the data) while `L2` shows the full bandwidth.

Why two HBM array sizes (134 MB and 536 MB)? The 134 MB size verifies that the workload is HBM-bound (overflows the 63 MB L2) but may not fully saturate the memory controllers. The 536 MB size provides a deeply saturated measurement where all HBM bandwidth is in use. Comparing the two shows whether the HBM controllers reach steady-state.

---

5. Comparison with CPU Machine File

The reference CPU file used for comparison is `Zen_Ryzen7-1700X.yml` (AMD Zen 1, a

typical kerncraft CPU machine file from the RRZE collection).

5.1 Top-level field comparison

Field	CPU file (Zen1)	GPU file (H100)	Status	Notes
`kerncraft version`	`0.8.11.dev0`	—	✗ absent	Not critical for roofline
`model name`	`AMD Ryzen 7 1700X`	`NVIDIA H100`	✓ present	
`model type`	`Intel Core ...`	`NVIDIA Hopper architecture`	✓ present	
`clock`	`3.6 GHz`	—	✗ absent	Fills from hardware_info
`sockets`	`1`	`1`	✓ present	
`cores per socket`	`8`	—	✗ absent	Replaced by `SMs: 132`
`SMs`	—	`132`	GPU-only	No CPU equivalent
`threads per core`	`2` (HT)	—	✗ absent	GPU uses warps per SM
`NUMA domains per socket`	`1`	—	✗ absent	GPU has no NUMA
`cores per NUMA domain`	`8`	—	✗ absent	
`transparent hugepage`	`always`	—	✗ absent	Not applicable to GPU
`in-core model` (OSACA)	`ZEN1`	`GPU: Hopper` (placeholder)	⚠ placeholder	No GPU backend in OSACA
`in-core model` (LLVM-MCA)	`-mcpu=znver1`	—	✗ absent	No GPU backend in LLVM-MCA
`isa`	`x86`	`PTX`	⚠ present	PTX not low-level enough
`FLOPs per cycle`	per core (DP/SP)	—	✗ absent	Replaced by `FLOPs per SM`
`FLOPs per SM`	—	`DP: {FMA:64,...}`	GPU-only	
`compiler`	`gcc`, `clang`, `icc`	—	✗ absent	Could add `nvcc`
`non-overlapping model`	port formula + port list	—	✗ absent	No GPU port model exists
`overlapping`	port formula +	—	✗ absent	Same reason

Field	CPU file (Zen1)	GPU file (H100)	Status	Notes
`model`	port list			
`cacheline size`	`64 B`	—	✗ absent	GPU L2 sector = 128 B
`compute capability`	—	`'9.0'`	GPU-only	
`warp size`	—	`32`	GPU-only	
`operation latencies`	—	FMA/DIV/SQRT	GPU-only	From cuda-incore
`roofline`	—	DP/SP/HP ceilings	GPU-only	From MixBench
`occupancy sweep`	—	init/read/scale/ triad	GPU-only	From gpu-stream

5.2 Memory hierarchy field comparison

Each level (L1, L2, L3/HBM, MEM) in the CPU file contains a rich set of fields. The table below shows which are present in the GPU file:

Field (per level)	CPU file	GPU file	Status	Notes
`level`	L1, L2, L3, MEM	L1, L2, HBM	✓ / ✗ L3	GPU has no L3
`cache per group.sets`	present	—	✗ absent	Nvidia does not publish
`cache per group.ways`	present	—	✗ absent	Same
`cache per group.cl_size`	`64 B`	—	✗ absent	Would be `128 B`
`cache per group.replacement_policy`	`LRU`	—	✗ absent	Hopper policy unpublished
`cache per group.write_allocate`	`true/false`	—	✗ absent	Unknown for GPU
`cache per group.write_back`	`true/false`	—	✗ absent	Unknown for GPU
`cache per group.load_from`	`L2`, `L3`	—	✗ absent	Could infer hierarchy
`cache per group.store_to`	`L2`, `L3`	—	✗ absent	
`cache per`	`L3`, `MEM`	—	✗ absent	Eviction policy

Field (per level)	CPU file	GPU file	Status	Notes
group.victims_to				unpublished
cores per group	1 (L1,L2), N (LLC)	—	✗ absent	Different GPU topology
groups	N cores	—	✗ absent	
performance counter metrics.loads	PAPI string	—	✗ absent	ncu strings available
performance counter metrics.stores	PAPI string	—	✗ absent	ncu strings available
performance counter metrics.misses	PAPI string	—	✗ absent	ncu strings available
performance counter metrics.evicts	PAPI string	—	✗ absent	ncu strings available
size per group	e.g. 32.00 kB	228.00 kB, 63 MB, ~	✓ present	
threads per group	based on HT	—	✗ absent	
upstream throughput	[arch, [ports]]	[description, duplex]	⚠ simplified	No port numbers
transfers overlap	true/false	—	✗ absent	
measured bandwidth	not in CPU files	33604 GB/s etc.	GPU-extra	Added for completeness
latency cycles	not in CPU files	32 cycles (L1)	GPU-extra	Added for completeness

5.3 Benchmark kernel definition comparison

Field	CPU file	GPU file	Status
FLOPs per iteration	✓ (0, 1, 2)	✓ (0, 1, 2)	✓ identical
fastest bench kernel	✓ e.g. load_avx	—	✗ absent
read streams	✓ {bytes, streams}	✓ {bytes, streams}	✓ identical

Field	CPU file	GPU file	Status
`read+write streams`	✓ `{bytes, streams}`	—	✗ absent
`write streams`	✓ `{bytes, streams}`	✓ `{bytes, streams}`	✓ present

5.4 Benchmark measurements structure comparison

This is the most important structural difference between the CPU and GPU machine files.

CPU file structure:

```
measurements:
  L2:               # cache level
    1:              # KEY = number of active cores
      cores: [1, 2, 3, 4, 5, 6, 7, 8]
      results:
        copy: [71.2 GB/s, 141.7 GB/s, 212.4 GB/s, ...]  # one value per core
count
      stats:
        copy:
        - [70.8, 71.0, 71.2, 70.9, ...]    # 10 repetitions at core count 1
        - [141.2, 141.5, 141.7, ...]        # 10 repetitions at core count 2
      size per core: [16.00 MB]
      size per thread: [8.00 MB]
      threads: [2, 4, 6, 8, 10, 12, 14, 16]
      threads per core: 2
      total size: [16.00 MB]
```

GPU file structure:

```
measurements:
  L2_4MB_block256:  # level + array size + block size in tag
    total size: 4.00 MB
    block size: 256   # KEY = block size (GPU equivalent of core count)
    grid size: 2048
    results:
      copy:
        HBM: [814.66, 732.74, 830.13, ...]  # raw repetitions (no max taken)
        L2:  [1628.22, 1464.49, ...]
```

Measurement field	CPU file	GPU file	Notes
Dimension key	core count (1…N)	block size (32…1024)	different meaning
`cores` list	all core counts	`[block_size]`	one block size per entry
`results`	one value per core count	raw list per sub-metric	GPU has HBM + L2 sub-keys
`stats`	10 raw reps per core count	embedded in `results`	different nesting
`size per core`	scaling with cores	fixed 4 or 536 MB	GPU has no per-SM allocation
`size per thread`	half of size/core (HT)	same as size/core	
`threads`	core_count × HT factor	`[block_size]`	

Measurement field	CPU file	GPU file	Notes
`threads per core`	2 (HT)	32 (warp size)	
`total size`	fixed	fixed	
Cache level tagging	separate L1/L2/L3/MEM keys	embedded in tag name	e.g. `L2_4MB_block256`
L1 measurements	✓ present	✗ absent	needs shared-memory kernels
L3 measurements	✓ present	✗ absent	GPU has no L3
HBM vs MEM	CPU has MEM	GPU has HBM sub-key	different naming
GFLOP/s sub-key	not in CPU files	✓ present (FLOPs>0 kernels)	GPU-extra

5.5 What the GPU file adds that CPU files do not have

The GPU machine file introduces several new fields that have no CPU equivalent but are necessary to describe GPU hardware correctly:

New GPU field	Value	Purpose
`compute capability`	`'9.0'`	Identifies GPU generation and feature set
`SMs`	`132`	SM count — the GPU "cores"
`FLOPs per SM`	`{FMA:64, ADD:64, ...}`	Per-SM compute throughput
`warp size`	`32`	Minimum unit of thread scheduling
`operation latencies`	FMA/DIV/SQRT in cycles	From cuda-incore; closest to in-core model
`roofline`	DP/SP/HP BW + compute	Explicit ceilings from MixBench
`occupancy sweep`	BW at 100% occupancy	From gpu-stream, shows GPU vs CPU scaling
`block size` in measurements	32 to 1024	Replaces core count as the scaling dimension
`HBM` sub-key in results	measured bytes	Hardware-verified via ncu counters
`L2` sub-key in results	measured bytes	Hardware-verified via ncu counters
`GFLOP/s` sub-key	measured compute	From ncu instruction counters
`measured bandwidth` in hierarchy	33604, 12759, 2229 GB/s	Explicit measured values per level
`latency cycles` in hierarchy	32, 250, 700 cycles	From gpu-latency benchmark

5.6 Summary: completeness relative to CPU file

Category	CPU fields total	GPU fields present	Coverage
Hardware topology	8	3 (model, type, SMs)	38%
In-core model	2 (OSACA + LLVM-MCA)	0 (impossible)	0%
Compiler flags	3 (gcc/clang/icc)	0	0%
Execution model (ports)	2 (non/overlapping)	0 (impossible)	0%
Memory hierarchy geometry	10 per level	2 per level (size, throughput)	20%
Performance counter metrics	4 per level	0 (ncu strings available but not added)	0%
Benchmark kernel definitions	5 fields per kernel	3 fields per kernel	60%
Benchmark measurements	8 fields per level	6 fields (different structure)	75%
Overall	**~80 distinct fields**	**~25 filled**	**~31%**

The 69% gap is almost entirely accounted for by two permanent limitations: the missing in-core model (requires OSACA/LLVM-MCA GPU support — impossible with current tools) and the missing cache geometry (Nvidia does not publish the H100 internal cache structure). The measurement section coverage is much higher at 75% because the ncu sweep provides the primary data kerncraft needs for roofline modelling.

---

6. Measurement Data — Full Results Tables

5.1 HBM 536 MB array — best bandwidth (GB/s) per kernel per block size

Kernel	block=32	block=64	block=128	block=256	block=512	block=1024
load	133	133	134	**134**	134	135
copy	783	1562	2162	**2177**	2168	2021
update	777	1550	2135	**2147**	2129	2010
daxpy	1182	2202	2230	2231	**2232**	2192
triad	1187	2217	2232	2235	**2238**	2197

Bold = peak value per kernel. Peak HBM bandwidth: **2395.87 GB/s** theoretical. Best measured: 2238 GB/s (triad, block=512) = **93.4% of theoretical peak**.

5.2 L2 4 MB array — best bandwidth (GB/s) per kernel per block size

Kernel	block=32	block=64	block=128	block=256	block=512	block=1024
load	125	124	124	**124**	124	124
copy	626	1024	1489	**1670**	1659	1589
update	629	1028	1481	**1670**	1659	1579
daxpy	934	1478	1890	**1890**	1890	1864
triad	934	1495	1796	**1900**	1881	1855

5.3 HBM 134 MB array — best bandwidth (GB/s) per kernel per block size

Kernel	block=32	block=64	block=128	block=256	block=512	block=1024
load	~125	~134	~134	**~134**	~134	~134
copy	~783	~1561	~2158	**~2175**	~2165	~2017
update	~777	~1550	~2131	**~2143**	~2126	~2006
daxpy	~1181	~2198	~2225	**~2229**	~2229	~2187
triad	~1185	~2213	~2228	**~2232**	~2236	~2194

5.4 Measurement reproducibility (HBM 536 MB, triad, block=256)

5 repetitions × 2 ncu passes = 10 data points:

```
[2234.37, 2234.92, 2229.88, 2228.62, 2230.83]
```

Mean = 2231.72 GB/s, Std = 2.79 GB/s, **CV = 0.12%**

This exceptionally low coefficient of variation confirms that ncu hardware counter measurements are highly reproducible on this H100 — far more stable than wall-clock timing.

---

7. Key Findings and Interpretation

Finding 1 — Block size is the critical dimension

The load kernel is nearly insensitive to block size (133–135 GB/s at HBM 536 MB across all configurations) because it is a reduction: it must process all elements sequentially at the warp level, and the bottleneck is always the memory bus rather than warp parallelism.

For copy, update, daxpy, and triad, block size has a dramatic effect:

- **Block=32** (one warp per block): copy achieves only 783 GB/s — 33% of peak. With one warp, only 32 threads issue memory transactions simultaneously. The H100 HBM controllers need hundreds of outstanding requests to saturate bandwidth. One warp cannot generate enough requests to hide the ~700-cycle HBM latency.

- **Block=64** (two warps per block): copy jumps to 1562 GB/s — 65% of peak. Two warps double the outstanding requests, halving the latency gap.
- **Block=128–256**: copy reaches 2162–2177 GB/s — 90–91% of peak. Latency is sufficiently hidden; further warps provide diminishing returns.
- **Block=512**: copy stays at 2168 GB/s — essentially the same as block=256. The memory controllers are saturated; extra threads do not help.
- **Block=1024**: copy drops slightly to 2021 GB/s — 84% of peak. With 1024 threads per block, each SM handles a very large block, reducing the number of active blocks per SM. This can reduce occupancy below the level needed for full bandwidth.

Practical implication: User code with block size ≤ 32 will achieve at most 33% of peak HBM bandwidth regardless of kernel efficiency. This is the key parameter the supervisor highlighted as missing from standard benchmarks.

Finding 2 — daxpy and triad saturate earlier than copy

At block=64, daxpy already reaches 2202 GB/s (92% of peak) while copy only reaches 1562 GB/s (65%). This is because daxpy and triad access more arrays per thread:

- copy: 1 read + 1 write = 16 bytes/element = 2 streams
- daxpy: 1 read + 1 read/write + 1 write = 24 bytes/element = 3 streams

More streams mean more outstanding HBM transactions per warp, hiding latency more effectively at lower occupancy.

Finding 3 — load kernel is anomalously slow regardless of block size

The load kernel (pure read, reduction) achieves only 133–135 GB/s even at full occupancy. This represents 5.6% of HBM bandwidth. The bottleneck is the warp-level reduction itself: the `atomicAdd` at the end of each warp serialises the final accumulation. This is expected behaviour for a global reduction and confirms the kernel is correctly implemented — it is measuring atomics throughput, not memory bandwidth.

Finding 4 — L2 measurements show bandwidth plateau at block=256

For L2-bound workloads (4 MB array), copy peaks at 1670 GB/s at block=256 and does not increase with larger blocks. This is because:

1. At 4 MB total working set, the data fits in the 63 MB shared L2.
2. At block=256, there are enough outstanding L2 requests to saturate the L2 → SM bandwidth (~12,731 GB/s aggregate), not just the HBM → L2 path.
3. The L2 bandwidth ceiling is lower than the combined SM bandwidth demand, so adding more threads beyond 256 does not help.

Finding 5 — ECC overhead is measurable

With ECC enabled, the theoretical peak HBM bandwidth of 2395.87 GB/s is reduced because ECC requires additional memory reads to compute and verify error-correction codes. Our best measured

bandwidth is 2238 GB/s — 93.4% of theoretical, consistent with the expected ~6% ECC overhead on HBM.

Finding 6 — L2 bytes exceed HBM bytes for HBM-bound workloads

For HBM 536 MB copy at block=256:

- HBM bytes: 2177 GB/s
- L2 bytes: 2226 GB/s

The L2 counter reads ~2% higher than the HBM counter. This is because the L2 includes cache-line fill traffic (the L2 reads full 128-byte cache lines from HBM even when the kernel only needs 8 bytes per element, though with streaming access patterns the GPU hardware should fetch full lines). The difference is within normal range.

---

8. Relationship to kerncraft

What kerncraft needs from a machine file

kerncraft supports two performance models:

1. **Roofline model**: needs peak bandwidth and peak compute per memory level
2. **ECM model**: additionally needs the in-core execution time from OSACA/LLVM-MCA

What this machine file provides

The GPU machine file supports the **roofline model only**. The ECM model is impossible for GPU because:

- OSACA has no GPU/PTX backend
- LLVM-MCA has no GPU backend
- Nvidia does not publish the Hopper SM execution port table
- SASS (the actual GPU assembly) is not publicly documented

For roofline modelling, this machine file provides:

- Peak HBM bandwidth: 2238 GB/s (measured), 2395.87 GB/s (theoretical)
- Peak L2 bandwidth: 12,759 GB/s (measured)
- Peak compute: 31,411 GFLOP/s DP, 59,970 GFLOP/s SP (MixBench)
- Arithmetic intensity of each kernel (from stream counts in kernels section)

The block size dimension — GPU analog of CPU core count

In the CPU machine file, the measurements section records bandwidth at each core count (1 core, 2 cores, ..., N cores). This captures how bandwidth scales as more cores participate. For CPUs, bandwidth typically scales linearly with cores until the memory controller saturates.

For GPU, all 132 SMs always participate. The scaling dimension is **block size** — it controls how many threads run per block, which determines occupancy (warps per SM) and therefore how effectively memory latency is hidden. The measurements section encodes this as:

```
L2_4MB_block32:   # 32 threads/block (low occupancy → low BW)
L2_4MB_block256:  # 256 threads/block (optimal → peak BW)
L2_4MB_block1024: # 1024 threads/block (over-committed → slight degradation)
```

This is the key adaptation needed to make GPU machine files meaningful for kerncraft: instead of asking "how does performance scale with core count?" the GPU question is "how does performance scale with block size?" The YAML structure is identical; only the interpretation of the key changes.

---

9. Limitations and Future Work

8.1 What is not in this file (and why)

Missing	**Reason**	**Fixability**
`in-core model` (OSACA/LLVM-MCA)	No GPU backend exists	Impossible with current tools
Port-based execution model	Nvidia does not publish Hopper port table	Impossible
ECM model data	Requires in-core model	Impossible
L1 measurements	Our kernels bypass L1 (stream to L2); L1 requires shared-memory kernels	Solvable
L2 latency	gpu-latency timed out before reaching L2 buffer sizes	Run with 3600s timeout
HBM latency	Partially captured: ~700 cycles at large buffers from gpu-latency	Fill manually
Exact L2 cache geometry	Nvidia does not publish H100 sets/ways	Accept approximation
L1/shared memory split	Configurable at runtime; not fixed	Run with multiple configs

8.2 Recommended future measurements

L1 measurements (highest priority): Write CUDA kernels that explicitly load data into shared memory and compute from it. This requires kernels of the form:

```
__global__ void kernel_l1_copy(const double* A, double* B, int n) {
    extern __shared__ double tile[];
    int tid = threadIdx.x;
    // Load into shared memory
    for (int i = tid; i < TILE; i += blockDim.x)
        tile[i] = A[blockIdx.x * TILE + i];
    __syncthreads();
    // Write from shared memory
```

```
    for (int i = tid; i < TILE; i += blockDim.x)
        B[blockIdx.x * TILE + i] = tile[i];
}
```

With TILE small enough to fit in each SM's 228 kB, this measures L1/shared memory bandwidth rather than L2 bandwidth, filling the L1 measurements section.

L2 latency:

```
timeout 3600 ./cuda-latency | tee gpu-latency-full.txt
# Then look for buffer sizes in the 500-5000 kB range
awk '$3 > 500 && $3 < 5000' gpu-latency-full.txt | head -20
```

Block size effects on roofline ridge point: The current roofline ceilings from MixBench used a fixed block size. Since block size affects how close kernels can get to the bandwidth ceiling, running MixBench with different block sizes would show whether the roofline itself is block-size-dependent (it is not — the ceiling is fixed by hardware; only efficiency toward it changes).

8.3 How to extend this file for kerncraft GPU support

For kerncraft to actually use GPU machine files, the following extensions would be needed in the kerncraft source code:

1. **Parser extension**: Add recognition of `SMs`, `block size`, and `HBM` level names
2. **Roofline GPU mode**: Use block size as the occupancy axis instead of core count
3. **Bandwidth selection**: For a given user kernel's block size, interpolate from the block size sweep to predict achieved bandwidth
4. **No ECM for GPU**: Flag GPU machine files as roofline-only, skip ECM path

These changes are the natural next step after establishing that the machine file format itself is correct and the measurement methodology is sound.

Appendix A — Complete Measurement Summary

HBM bandwidth — all configurations (GB/s, best of 10 measurements)

Configuration	load	copy	update	daxpy	triad
HBM 536MB block32	133	783	777	1182	1187
HBM 536MB block64	133	1562	1550	2202	2217
HBM 536MB block128	134	2162	2135	2230	2232
HBM 536MB block256	134	**2177**	**2147**	2232	2235
HBM 536MB block512	134	2168	2129	2232	**2238**

Configuration	load	copy	update	daxpy	triad
HBM 536MB block1024	**135**	2021	2010	2192	2197
HBM 134MB block32	125	783	777	1182	1187
HBM 134MB block64	134	1562	1550	2199	2214
HBM 134MB block128	134	2162	2133	2228	2231
HBM 134MB block256	134	2177	2145	2231	2235
HBM 134MB block512	134	2168	2128	2231	2237
HBM 134MB block1024	135	2020	2009	2191	2196

L2 bandwidth — all configurations (GB/s, best of 10 measurements)

Configuration	load	copy	update	daxpy	triad
L2 4MB block32	125	626	629	934	934
L2 4MB block64	124	1024	1028	1478	1495
L2 4MB block128	124	1489	1481	1890	1796
L2 4MB block256	124	**1670**	**1670**	**1890**	**1900**
L2 4MB block512	124	1659	1659	1890	1881
L2 4MB block1024	124	1589	1579	1864	1855

Appendix B — Memory Hierarchy Reference Values

Parameter	Value	Source
L1 aggregate bandwidth	33,604 GB/s	gpu-benches/gpu-cache
L1 bandwidth per SM	~255 GB/s	gpu-cache ÷ 132 SMs
L1 latency	32 cycles	gpu-benches/gpu-latency
L2 peak bandwidth	12,759 GB/s	gpu-benches/gpu-l2-cache
L2 effective size	~39.5 MB	gpu-l2-cache transition

Parameter	Value	Source
L2 latency	~250 cycles	estimated (L2 typical for Hopper)
HBM sustained bandwidth	2,229 GB/s	gpu-l2-cache plateau
HBM latency	~700 cycles	gpu-latency tail (large buffers)
HBM theoretical peak	2,395.87 GB/s	hardware calculation
PCIe bandwidth	56.38 GB/s	cuda-memcpy

Appendix C — Operation Latencies (from cuda-incore, gpu-benches)

Measured with ILP=1 (fully sequential dependency chain), TLP=1 warp:

Operation	Latency (cycles)	Peak throughput
FMA (type a)	4.1 cycles	~4 ops/cycle/SM
DIV (type a)	166 cycles	pipeline-limited
SQRT (type a)	134 cycles	pipeline-limited
FMA (type b)	8.1 cycles	~2 ops/cycle/SM
DIV (type b)	136 cycles	pipeline-limited
SQRT (type b)	103 cycles	pipeline-limited

The two precision types (a and b) correspond to the two sets of 6×4 matrices output by cuda-incore. Type a has 4-cycle FMA latency (consistent with single-precision FP32 on Hopper). Type b has 8-cycle FMA latency (consistent with double-precision FP64 on Hopper).

Appendix D — Cross-Validation of HBM Bandwidth

Four independent tools measured HBM bandwidth for comparable workloads:

Tool	Kernel	Config	Measured BW
Custom ncu	triad	block=512, 536 MB	2238 GB/s
Custom ncu	copy	block=256, 536 MB	2177 GB/s
BabelStream	Triad	block=256 (fixed)	2193 GB/s
BabelStream	Copy	block=256 (fixed)	2167 GB/s
MixBench	DP roofline	fixed config	2150 GB/s
gpu-stream	triad	100% occupancy	2156 GB/s
gpu-l2-cache	sustained	large buffers	2230 GB/s
Theoretical	—	—	2396 GB/s

All four tools agree within 4% on HBM bandwidth. This cross-validation confirms the measurement methodology is sound and the values in the machine file are reliable.